Employee Attrition Prediction Mod

Version 1.0

Developer Documentation · ML Pipeline & Inference Guide

Project	Employee Attrition Prediction Model
Version	1.0
Problem Type	Binary Classification — Will an employee leave? (Yes / No)
Final Model	Logistic Regression with GridSearchCV + SMOTE (hyper-tuned)
Dataset	IBM HR Analytics Dataset — 1,470 employees, 35 raw features
Purpose	Handover documentation for the next developer / data scientist
Date	May 2026

This document is the complete technical reference for the Employee Attrition Prediction ML project. It covers the full pipeline from raw data to production inference — including data cleaning, feature engineering, model selection, hyperparameter tuning, threshold optimisation, saved artefacts, and the explainable inference system. Everything a new developer needs to understand, reproduce, extend, or integrate this model.

Table of Contents

1 Project Overview & Problem Statement

Employee attrition — voluntary turnover — is one of the costliest HR challenges. Replacing a single employee typically costs 50–200% of their annual salary when accounting for recruitment, onboarding, and lost productivity. This project builds a **binary classification model** that predicts whether a given employee is likely to leave the organisation, and explains *why* using model-driven contribution scores — enabling HR teams to take targeted retention action before an employee resigns.

Business Goals

- **Predict attrition risk** — classify each employee as Likely to Leave or Likely to Stay
- **Explain the prediction** — surface the top risk drivers and protective factors using logistic regression coefficients
- **Suggest actions** — map each risk factor to a concrete HR retention action
- **Minimise false negatives** — the model is tuned to maximise recall (catch as many leavers as possible) while maintaining acceptable precision

Key Design Decisions

- **Logistic Regression chosen as final model** — despite tree-based models achieving similar accuracy, LR provides coefficient-level interpretability essential for the explainability feature
- **SMOTE for class imbalance** — the dataset has ~16% attrition (minority class); SMOTE oversamples the minority class during training
- **Custom threshold tuning** — default 0.5 threshold is replaced with an optimised threshold that maximises F1 while keeping recall above 0.55
- **Engineered features replace raw ones** — 10 raw columns are dropped; 7 engineered ratio features are created that better capture latent risk signals

2 Repository Structure

File / Folder	Type	Role
Raw Dataset.csv	Input data	Original IBM HR dataset — 1,470 rows, 35 columns
Cleaned_Dataset.csv	Processed data	After dropping redundant columns + adding engineered features (LeaveFrequency, ManagerFeedback)
clean_dataset.ipynb	Notebook — Stage 1	Data cleaning, EDA visualisations, initial feature engineering
algo_choosing.ipynb	Notebook — Stage 2	Trains 8 classifiers, evaluates all metrics, saves model_comparison.csv
hypertuned.ipynb	Notebook — Stage 3	GridSearchCV on Logistic Regression, threshold tuning, saves all production artefacts
test.ipynb	Notebook — Stage 4	Production inference script with explainability and action recommendation
attrition_model.pkl	Saved artefact	Full sklearn Pipeline (preprocessor + SMOTE + LR model) serialised with joblib
columns.json	Saved artefact	Ordered list of 29 feature columns the model expects as input
threshold.json	Saved artefact	Optimised classification threshold (0.50)
metrics.json	Saved artefact	Final production metrics: accuracy, precision, recall, F1, ROC-AUC
model_comparison.csv	Output	All 8 model results for algorithm selection reference

3 ML Pipeline Overview

The project follows a standard supervised ML lifecycle with five sequential stages. Each stage is contained in its own notebook for clarity and reproducibility.

Stage 1: Data Cleaning	**Notebook:** clean_dataset.ipynb Load raw CSV. Drop constant/leaky columns (EmployeeCount, Over18, StandardHours, EmployeeNumber). Encode target (Attrition: Yes→1, No→0). Engineer LeaveFrequency and ManagerFeedback. Perform EDA (distribution plots, heatmap, box plots). Save Cleaned_Dataset.csv.
Stage 2: Feature Engineering	**Notebook:** algo_choosing.ipynb (top) Load Cleaned_Dataset.csv. Engineer 7 ratio features (SalaryGrowth, PromotionDelay, WorkPressure, CareerStagnation, Stability, IncomePerLevel, CommuteStress). Drop 10 raw columns that are now represented by engineered features.
Stage 3: Algorithm Selection	**Notebook:** algo_choosing.ipynb 80/20 stratified train-test split (random_state=42). Build sklearn Pipeline: OneHotEncoder for 6 categorical columns + StandardScaler for numeric columns + SMOTE + Classifier. Train 8 models. Record Train Accuracy, Test Accuracy, Precision, Recall, F1, ROC-AUC. Save model_comparison.csv.
Stage 4: Hyperparameter Tuning	**Notebook:** hypertuned.ipynb Select Logistic Regression (best interpretability + ROC-AUC). GridSearchCV over C (10 log-spaced values), penalty (l1/l2), solver (liblinear/saga), class_weight (None/balanced). 5-fold CV, scoring=f1. Tune classification threshold from 0.50 to 0.70 (step 0.01), keeping recall >= 0.55. Save model PKL, columns JSON, threshold JSON, metrics JSON.
Stage 5: Inference & Explainability	**Notebook:** test.ipynb Load artefacts. Run preprocessor transform on input. Multiply transformed values by logistic coefficients to get per-feature contribution scores. Rank into risk drivers (positive contribution) and protective factors (negative contribution). Map to human-readable labels and suggested HR actions. Print structured report.

4 Stage 1 — Data Cleaning (clean_dataset.ipynb)

Input: Raw Dataset.csv (1,470 rows × 35 columns)

Output: Cleaned_Dataset.csv (1,470 rows × 33 columns)

Columns Dropped

Column	Reason for Dropping
EmployeeCount	Constant value (1 for all rows) — zero variance
Over18	Constant value ('Y' for all rows) — zero variance
StandardHours	Constant value (80 for all rows) — zero variance
EmployeeNumber	Unique identifier — leaky, not predictive

Target Encoding

The Attrition column is label-encoded: **Yes** → **1** (left), **No** → **0** (stayed). The dataset is imbalanced: approximately **84% Stay (0)** vs **16% Leave (1)**. This imbalance is addressed later by SMOTE during training.

Initial Feature Engineering in clean_dataset.ipynb

Feature	Formula	Business Rationale
LeaveFrequency	NumCompaniesWorked / (YearsAtCompany + 1), normalised to [0,1]	High job-hopping behaviour: many companies in fewer years → higher leave frequency → higher attrition risk
ManagerFeedback	(JobSatisfaction + RelationshipSatisfaction + EnvironmentSatisfaction) / 3, normalised to [0,1]	Composite proxy for employee-manager relationship quality; low scores → dissatisfaction → higher risk

EDA Insights from Visualisations

Attrition Distribution	Severe class imbalance (~16% leave). This motivated SMOTE oversampling strategy.
Age vs Attrition (Box Plot)	Younger employees (mid-20s to early 30s) show higher attrition — career exploration phase.
Income vs Attrition (Box Plot)	Lower monthly income employees leave more — financial dissatisfaction is a key signal.

Overtime vs Attrition (Count Plot)	Employees who work overtime leave at significantly higher rates than those who do not.
Job Satisfaction vs Attrition	Employees with satisfaction score 1 (lowest) leave at the highest rate.
Feature Correlation Heatmap	MonthlyIncome, JobLevel, TotalWorkingYears, and YearsAtCompany are highly correlated — captured via engineered ratios in Stage 2.

5 Stage 2 — Feature Engineering

Seven additional ratio features are engineered from the cleaned dataset. These replace 10 raw columns that either encode the same information redundantly or are better represented as ratios normalised by tenure. **This engineering step runs in algo_choosing.ipynb and hypertuned.ipynb — and must be replicated exactly in test.ipynb at inference time.**

Engineered Feature	Formula	Dropped Raw Columns	Business Meaning
SalaryGrowth	PercentSalaryHike / (YearsAtCompany + 1)	PercentSalaryHike, YearsAtCompany	Salary growth rate per year of tenure — low values indicate stagnant compensation
PromotionDelay	YearsSinceLastPromotion / (YearsAtCompany + 1)	YearsSinceLastPromotion, YearsAtCompany	Proportion of tenure since last promotion — high values indicate career stagnation
WorkPressure	OverTime × (4 – WorkLifeBalance)	OverTime, WorkLifeBalance	Compound stress: overtime combined with poor work-life balance amplifies burnout risk
CareerStagnation	(YearsInCurrentRole + YearsSinceLastPromotion) / (TotalWorkingYears + 1)	YearsInCurrentRole, TotalWorkingYears	Proportion of career spent without role progression — high = stuck
Stability	YearsAtCompany / (TotalWorkingYears + 1)	YearsAtCompany, TotalWorkingYears	Loyalty metric — high = spent most of career at this company
IncomePerLevel	MonthlyIncome / (JobLevel + 1)	MonthlyIncome, JobLevel	Compensation relative to seniority — low = underpaid for the level
CommuteStress	DistanceFromHome × OverTime	DistanceFromHome	Long commute combined with overtime — compounding dissatisfaction factor

Critical note: The engineer_features() function in test.ipynb must exactly match the feature engineering code in algo_choosing.ipynb and hypertuned.ipynb. Any discrepancy will cause silent prediction errors because the model was trained on these specific transformations.

6 Stage 3 — Algorithm Selection (algo_choosing.ipynb)

Eight classifiers are benchmarked under identical conditions: same pipeline (OneHotEncoder + StandardScaler + SMOTE + Classifier), same train/test split (80/20, stratified, random_state=42), same evaluation metrics.

Pipeline Architecture

Step	Class	Description
preprocessor	make_column_transformer	OneHotEncoder (handle_unknown='ignore') on 6 categorical columns; StandardScaler on all numeric columns
smote	SMOTE(random_state=42)	Synthetic Minority Oversampling — applied only to training data inside the pipeline to prevent data leakage
model	Various classifiers	8 classifiers tested; each wrapped identically

Categorical Columns (OneHotEncoded)

BusinessTravel, Department, EducationField, Gender, JobRole, MaritalStatus

Model Comparison Results

Model	Train Acc	Test Acc	Precision	Recall	F1	ROC-AUC
Logistic Regression	79.3%	78.2%	0.392	0.660	0.492	0.793
SVM	96.2%	85.0%	0.538	0.447	0.488	0.744
XGBoost	96.9%	86.1%	0.625	0.319	0.423	0.771
LightGBM	96.3%	86.1%	0.625	0.319	0.423	0.767
Gradient Boosting	99.7%	85.4%	0.577	0.319	0.411	0.768
Random Forest	98.3%	85.4%	0.583	0.298	0.394	0.777
Decision Tree	83.3%	77.2%	0.328	0.404	0.362	0.736
KNN	100%	55.8%	0.218	0.681	0.330	0.669

Model Selection Rationale

Logistic Regression was selected as the final model — not because it has the highest accuracy (XGBoost/LightGBM marginally win on test accuracy), but for two decisive reasons:

• **Interpretability:** Logistic regression coefficients directly quantify each feature's contribution to the prediction probability. This is the foundation of the contribution scoring system in test.ipynb — each prediction comes with a model-driven explanation.

• **Best Recall:** With 0.660, Logistic Regression catches the most actual leavers. In attrition prediction, a false negative (missing someone who will leave) is far more costly than a false positive (flagging someone who stays). Recall is the primary business metric.

• **No overfitting:** Tree-based models show train accuracy 96-100% vs test accuracy 85% — clear overfitting. LR's train/test gap is minimal (79.3% vs 78.2%).

• **Hyperparameter tunability:** LR's C parameter, penalty, and solver are well-understood and can be precisely optimised via GridSearchCV.

7 Stage 4 — Hyperparameter Tuning (hypertuned.ipynb)

GridSearchCV Configuration

Parameter	Values	Description
model__C	np.logspace(-3, 2, 10)	10 values from 0.001 to 100 — regularisation strength
model__penalty	['l1', 'l2']	L1 (Lasso, sparse) vs L2 (Ridge, dense) regularisation
model__solver	['liblinear', 'saga']	Both support L1; saga supports larger datasets
model__class_weight	[None, 'balanced']	Balanced automatically adjusts for class imbalance
cv	5	5-fold cross-validation
scoring	'f1'	Optimise for F1 — balances precision and recall
n_jobs	-1	Use all available CPU cores

SMOTE Adjustment

In the tuning stage, SMOTE uses **sampling_strategy=0.6** instead of the default 1.0. This means the minority class (leavers) is oversampled to 60% of the majority class size — a softer oversample that reduces the risk of overfitting to synthetic samples while still addressing imbalance.

Threshold Optimisation

After fitting the best model, the classification threshold is swept from 0.50 to 0.70 in 0.01 increments. The best threshold is chosen as the one that maximises F1 score while keeping recall above 0.55. This ensures the model doesn't sacrifice too many true positives in pursuit of precision.

```
for t in np.arange(0.50, 0.70, 0.01):
y_pred = (y_prob >= t).astype(int)
if f1_score > best_f1 and recall_score > 0.55:
best_threshold = t
```

Final Production Metrics

Metric	Value	Interpretation
Accuracy	86.05%	Overall correctness across both classes

Precision	56.52%	Of employees flagged as leaving, 56.5% actually left
Recall	55.32%	Of employees who actually left, 55.3% were correctly flagged
F1 Score	55.91%	Harmonic mean of precision and recall
ROC-AUC	80.51%	Model's ability to rank leavers above stayers (80.5% vs 50% random)
Threshold	0.50	Probability cutoff: predict Leave if P(leave) >= 0.50

Saved Artefacts

• **attrition_model.pkl** — complete Pipeline (preprocessor + SMOTE + best LR model) serialised via joblib

• **columns.json** — ordered list of 29 feature names the model expects; used to reindex input DataFrames at inference time

• **threshold.json** — {"threshold": 0.50} — the optimised decision boundary

• **metrics.json** — all 5 production metrics for monitoring and regression testing

8 Stage 5 — Inference & Explainability (test.ipynb)

The inference script loads the saved artefacts and exposes a predict() function that takes a single employee record (as a Python dict) and outputs a full attrition risk report with model-driven explanations and HR action recommendations.

Prediction Pipeline (step by step)

Step	Detail
1. Load artefacts	joblib.load('attrition_model.pkl'), threshold.json, columns.json
2. Extract internals	preprocessor = model.named_steps['preprocessor']; logistic_model = model.named_steps['model']; COEFS = logistic_model.coef_[0]
3. Feature engineering	engineer_features(df) — must exactly match training code
4. Column alignment	df.reindex(columns=columns, fill_value=0) — ensures feature order matches training
5. Probability	prob = model.predict_proba(df_aligned)[:, 1][0]
6. Prediction	pred = int(prob >= threshold)
7. Contribution scoring	X_transformed = preprocessor.transform(df_aligned); contributions = X_transformed[0] * COEFS
8. Risk ranking	Sort positive contributions (risk drivers) and negative contributions (protective factors)
9. Label mapping	Map internal feature names to human-readable labels via LABEL_MAP
10. Action mapping	Map top risk driver label to retention action via ACTION_MAP
11. Output report	Print structured report with risk %, prediction, reason, action, ranked drivers

Contribution Scoring — How It Works

Logistic regression predicts P(leave) = sigmoid(w·x + b). The contribution of feature i is defined as **contribution_i = x_i_transformed × w_i**. Features with **positive contribution push the model toward predicting Leave**; features with **negative contribution push toward Stay**. Only contributions above an

absolute threshold of 0.05 are reported to filter noise.

LABEL_MAP — Feature Name Translations

The LABEL_MAP dictionary translates sklearn internal feature names to business-readable strings:

Internal Feature Name	Human-Readable Label
standardscaler__WorkPressure	High overtime / poor work-life balance
standardscaler__PromotionDelay	Promotion delay
standardscaler__CareerStagnation	Career stagnation
standardscaler__IncomePerLevel	Low income for job level
standardscaler__LeaveFrequency	High leave frequency
standardscaler__Stability	Long company tenure
onehotencoder__MaritalStatus_Single	Single (higher mobility)
onehotencoder__BusinessTravel_Travel_Frequently	Frequent business travel
onehotencoder__Department_Sales	Sales department

ACTION_MAP — Retention Recommendations

Each risk label maps to a concrete HR retention action:

Risk Label	Suggested HR Action
High overtime / poor work-life balance	Discuss workload redistribution and offer flexible working hours
Promotion delay	Schedule a career growth discussion and define a promotion timeline
Low salary growth	Review compensation and benchmark against market rates
Career stagnation	Offer new responsibilities, lateral moves, or upskilling programs
Low manager feedback	Facilitate regular 1:1s and manager-employee alignment sessions
High commute stress with overtime	Explore remote work or hybrid arrangements
No / low stock options	Consider adding stock options or a long-term incentive plan

Single (higher mobility)	Strengthen engagement through mentorship and team bonding
Low job involvement	Assign high-impact projects to increase ownership and engagement
Frequent business travel	Review travel load and explore remote-work alternatives

9 Saved Artefacts Reference

attrition_model.pkl

A joblib-serialised sklearn/imbalanced-learn Pipeline with 3 named steps:

Step Name	Object	Purpose
preprocessor	ColumnTransformer	OneHotEncoder (6 cat cols) + StandardScaler (23 num cols) → 49 transformed features
smote	SMOTE(random_state=42, sampling_strategy=0.6)	Oversamples minority class during fit; skipped at predict_proba time
model	LogisticRegression (best params from GridSearchCV)	Final classifier; coef_ array drives explainability

Load example:

```
import joblib
model = joblib.load('attrition_model.pkl')
preprocessor = model.named_steps['preprocessor']
lr = model.named_steps['model']
```

columns.json

An ordered JSON array of 29 feature names that the model expects as input columns. These are the post-engineering, pre-preprocessing column names. The inference script calls df.reindex(columns=columns, fill_value=0) to ensure correct column ordering before passing to the pipeline.

```
["Age", "BusinessTravel", "DailyRate", "Department", "Education",
"EducationField", "EnvironmentSatisfaction", "Gender", "HourlyRate",
"JobInvolvement", "JobRole", "JobSatisfaction", "MaritalStatus",
"MonthlyRate", "NumCompaniesWorked", "PerformanceRating",
"RelationshipSatisfaction", "StockOptionLevel", "TrainingTimesLastYear",
"YearsWithCurrManager", "LeaveFrequency", "ManagerFeedback",
"SalaryGrowth", "PromotionDelay", "WorkPressure",
"CareerStagnation", "Stability", "IncomePerLevel", "CommuteStress"]
```

threshold.json

```
{"threshold": 0.5}
```

The optimised decision boundary. Change this value to adjust the precision/recall trade-off without retraining.

metrics.json


```
{"accuracy": 0.8605, "precision": 0.5652, "recall": 0.5532,
"f1": 0.5591, "roc_auc": 0.8051}
```


Use for baseline comparison when retraining or evaluating model drift.

10 Dataset Schema — Raw & Engineered Features

Raw Dataset Columns (35 total)

Column	Type	Description
Age	int	Employee age
Attrition	str→int	Target: Yes/No → 1/0
BusinessTravel	str (cat)	Non-Travel / Travel_Rarely / Travel_Frequently
DailyRate	int	Daily wage rate
Department	str (cat)	Sales / Research & Development / Human Resources
DistanceFromHome	int	Commute distance in km — dropped after engineering
Education	int (1-5)	Education level ordinal
EducationField	str (cat)	Field of study
EmployeeCount	int	Always 1 — DROPPED (constant)
EmployeeNumber	int	Unique ID — DROPPED (leaky)
EnvironmentSatisfaction	int (1-4)	Workplace satisfaction ordinal
Gender	str (cat)	Male / Female
HourlyRate	int	Hourly wage
JobInvolvement	int (1-4)	Level of job involvement
JobLevel	int (1-5)	Seniority level — dropped after engineering
JobRole	str (cat)	9 distinct roles
JobSatisfaction	int (1-4)	Job satisfaction ordinal
MaritalStatus	str (cat)	Single / Married / Divorced
MonthlyIncome	int	Monthly salary — dropped after engineering
MonthlyRate	int	Monthly rate
NumCompaniesWorked	int	Number of previous employers

Over18	str	Always 'Y' — DROPPED (constant)
OverTime	str→int	Yes/No → 1/0 — dropped after engineering
PercentSalaryHike	int	Last salary hike % — dropped after engineering
PerformanceRating	int (1-4)	Performance rating
RelationshipSatisfaction	int (1-4)	Relationship satisfaction with manager/team
StandardHours	int	Always 80 — DROPPED (constant)
StockOptionLevel	int (0-3)	Stock option grant level
TotalWorkingYears	int	Total career experience — dropped after engineering
TrainingTimesLastYear	int	Training sessions attended
WorkLifeBalance	int (1-4)	Work-life balance self-rating — dropped after engineering
YearsAtCompany	int	Tenure at current company — dropped after engineering
YearsInCurrentRole	int	Years in current role — dropped after engineering
YearsSinceLastPromotion	int	Years since last promotion — dropped after engineering
YearsWithCurrManager	int	Years working with current manager

11 Model Performance Summary

Model	Test Acc	Precision	Recall	F1	ROC-AUC	Notes
Logistic Regression	78.2%	39.2%	66.0%	49.2%	79.3%	After feature engineering + SMOTE — pre-tuning
LR (Hyper-tuned)	86.1%	56.5%	55.3%	55.9%	80.5%	GridSearchCV + threshold optimisation — PRODUCTION
XGBoost (reference)	86.1%	62.5%	31.9%	42.3%	77.1%	Higher precision but much lower recall
SVM (reference)	85.0%	53.8%	44.7%	48.8%	74.4%	Similar F1 to baseline LR, no interpretability

Interpreting the Metrics in Context

• **Accuracy (86%)** — The dataset is 84% negative class; a dummy 'always stay' model would achieve 84% accuracy. Our model adds genuine value through its recall and ROC-AUC.

• **Recall (55.3%)** — The model identifies 55 out of every 100 employees who will actually leave. This is the most business-critical metric — missed leavers (false negatives) represent lost retention opportunities.

• **Precision (56.5%)** — Of employees flagged as at-risk, about 56% will actually leave. The remaining 44% will stay but still benefit from the recommended HR engagement actions.

• **ROC-AUC (80.5%)** — The model ranks a randomly chosen leaver above a randomly chosen stayer 80.5% of the time (vs 50% chance). Strong discriminative ability.

• **F1 (55.9%)** — Balanced metric reflecting the precision-recall trade-off under class imbalance.

12 How to Run the Project

Prerequisites

Python 3.9+ with the following packages:

```
pip install pandas numpy scikit-learn imbalanced-learn xgboost lightgbm joblib
```

Step 1: Data Cleaning (optional — Cleaned_Dataset.csv is already provided)

- Open clean_dataset.ipynb and run all cells.
- Update the CSV path if needed (currently hardcoded to Windows path).
- Output: Cleaned_Dataset.csv

Step 2: Algorithm Selection (optional — model_comparison.csv is already provided)

- Open algo_choosing.ipynb and run all cells.
- Reads Cleaned_Dataset.csv, trains 8 models, saves model_comparison.csv.
- Review the comparison table to confirm algorithm choice.

Step 3: Hyperparameter Tuning (optional — attrition_model.pkl is already provided)

- Open hypertuned.ipynb and run all cells.
- Reads Cleaned_Dataset.csv, runs GridSearchCV (~5-15 min depending on hardware).
- Saves: attrition_model.pkl, columns.json, threshold.json, metrics.json

Step 4: Run Inference

- Open test.ipynb and run all cells.
- All 4 artefacts must be in the same directory.
- Modify the test cases at the bottom to predict for your own employee records.

Predicting for a Single Employee

Call predict() with a dictionary containing the raw feature values:

```
predict({
"BusinessTravel": "Travel_Frequently",
"Department": "Sales",
"EducationField": "Marketing",
"Gender": "Male",
"JobRole": "Sales Executive",
"MaritalStatus": "Single",
"Age": 26, "DailyRate": 400, "Education": 2,
"EnvironmentSatisfaction": 1, "HourlyRate": 45,
# ... all raw feature values ...
```

```
}, name="Employee Name | Role")
```

Sample Output (High Risk Employee)

```
============================================================
Employee : Arjun Singh | Sales Executive
============================================================
Attrition Risk : 73.4%
Prediction : LIKELY TO LEAVE
Reason : High overtime / poor work-life balance
+ Promotion delay
Suggested Action : Discuss workload redistribution and
offer flexible working hours

Risk Drivers (contribution score):
[+0.412] ████████ High overtime / poor work-life balance
[+0.298] ██████ Promotion delay
[+0.187] ████ Frequent business travel

Protective Factors:
[-0.112] ██ Long company tenure
```

13 Integration Guide

Integrating into a Web API (FastAPI / Flask)

The inference logic in test.ipynb can be converted to a REST API in three steps:

- **Extract functions** — copy engineer_features(), get_contributions(), and predict() into a module (e.g. predictor.py). Load artefacts once at startup.
- **Accept raw employee JSON** — the API endpoint receives the same dict structure that predict() expects. Validate required fields server-side.
- **Return structured JSON** — instead of printing, return: {risk_pct, prediction, reason, action, risk_drivers[], protective_factors[]}

Integrating with the AI HR Chatbot

This model integrates naturally with the AI HR Chatbot project. The chatbot's tools.py can be extended with an attrition_risk_tool that:

- Queries MongoDB for the employee's raw feature values
- Calls engineer_features() + model.predict_proba()
- Returns the risk percentage, top reason, and suggested action as a formatted string
- The HR chatbot LLM then presents this in a conversational format to an HR manager

Batch Prediction (All Employees)

```
df_all = pd.read_csv('Cleaned_Dataset.csv')
df_eng = engineer_features(df_all.drop('Attrition', axis=1))
df_ali = df_eng.reindex(columns=columns, fill_value=0)
probs = model.predict_proba(df_ali)[:, 1]
df_all['AttritionRisk'] = probs
df_all['AttritionFlag'] = (probs >= threshold).astype(int)
df_all.sort_values('AttritionRisk', ascending=False).head(20)
```

14 Extension Guide

Adding a New Feature	1. Add the engineering logic to the engineer_features() function in test.ipynb (and the corresponding training notebooks). 2. Add the feature name to LABEL_MAP with a human-readable label. 3. Optionally add an entry to ACTION_MAP. 4. Retrain the model (hypertuned.ipynb) and save new artefacts.
Retraining with New Data	Add new employee records to Cleaned_Dataset.csv (or re-run clean_dataset.ipynb on updated raw data). Re-run hypertuned.ipynb — GridSearchCV will find new optimal hyperparameters. Compare new metrics.json against the old one to confirm improvement or catch regression.
Switching to a Different Model	Replace the LogisticRegression step in hypertuned.ipynb with any sklearn-compatible classifier. Note: only linear models (LR, LinearSVC) support direct coefficient-based contribution scoring. For tree-based models, use SHAP (pip install shap) for equivalent explainability.
Adjusting Precision/Recall Trade-off	Edit threshold.json: lower the threshold (e.g. 0.35) to catch more leavers at the cost of more false alarms. Raise it (e.g. 0.65) for higher precision. No retraining required — just update the JSON file.
Adding SHAP Explainability	Install shap. Use shap.LinearExplainer(lr, X_background) for logistic regression. SHAP values are a more rigorous alternative to coefficient × value contribution scoring, accounting for feature correlations.

15 Troubleshooting

Problem	FileNotFoundError: attrition_model.pkl
Solution	Run hypertuned.ipynb first to generate the model file. All 4 artefact files must be in the same directory as test.ipynb.
Problem	ValueError: feature names mismatch
Solution	The columns passed at inference don't match what the model was trained on. Ensure engineer_features() in test.ipynb exactly matches the training code. Use df.reindex(columns=columns, fill_value=0) before predict_proba().
Problem	All predictions are 'LIKELY TO STAY' (no attrition detected)
Solution	The threshold may be too high for your data distribution. Lower the threshold in threshold.json (e.g. from 0.50 to 0.35) and test again. Also verify SMOTE ran correctly during training.
Problem	GridSearchCV takes too long
Solution	Reduce the param grid: use fewer C values (e.g. [0.01, 0.1, 1.0, 10]) or limit cv=3. Alternatively, use RandomizedSearchCV with n_iter=20.
Problem	ModuleNotFoundError: imbalanced-learn
Solution	pip install imbalanced-learn. Note: imblearn is installed separately from sklearn.
Problem	ModuleNotFoundError: xgboost / lightgbm
Solution	pip install xgboost lightgbm. These are only needed for algo_choosing.ipynb; test.ipynb (inference) does not require them.
Problem	Contribution scores all near zero
Solution	The input values may not have been feature-engineered before passing to the preprocessor. Always call engineer_features(df) before reindex() and predict_proba().

Problem	SettingWithCopyWarning in feature engineering
Solution	Use df = df.copy() at the start of engineer_features() to ensure you're working on a copy. The code already does this.

End of Documentation

This document covers the complete Employee Attrition Prediction Model as of May 2026.
For questions, refer to the notebooks or contact the original author.

Built with: Python · scikit-learn · imbalanced-learn · XGBoost · LightGBM · joblib